

Timeout
 |
 Gossip Confirma Falha
 |
 Bully Election
 |
 Novo Coordenador
 |
 Carrega Checkpoint
 |
 Aplica Logs
 |
 Incremental State Transfer
 |
 Sistema Operacional

INSTRUÇÕES E METODOLOGIA PARA O DESENVOLVIMENTO DO TRABALHO

- 1) O trabalho deverá ser feito em dupla;
- 2) Pode usar IA;
- 3) O tempo de duração do trabalho é de 8 semanas.
- 4) A cada checkpoint uma parte do trabalho deverá ser apresentada. As datas de checkpoint são: 18/09, 30/09, 09/10, 21/10, 30/10 e 06/11.
- 5) Como são dois alunos, a divisão do trabalho será de acordo com a tabela abaixo para que não sobrecarregue um aluno apenas. E, desta forma, no dia do checkpoint, apenas o aluno responsável pela sua parte deverá apresentar.

Área	Aluno 1	Aluno 2
Protocolo de mensagens	Responsável	Apoio
TCP/Sockets	Responsável	Apoio
Cliente P2P	Responsável	—
Upload/download	Responsável	Apoio
Fragmentação	Responsável	—
SHA-256	Responsável	—
LZ4	Responsável	—
Armazenamento de arquivos	Responsável	—
LFU Cache	Responsável	—
Metadata	Apoio	Responsável
Hash table	Apoio	Responsável
Super Peer	Apoio	Responsável

Área	Aluno 1	Aluno 2
Chord	—	Responsável
Finger Table	—	Responsável
Gossip	—	Responsável
Heartbeat	—	Responsável
Detecção de falhas	—	Responsável
Bully	—	Responsável
SMR	—	Responsável
2PC	—	Responsável
IST	—	Responsável
Integração	50%	50%
Testes	50%	50%
Documentação	50%	50%

6) CHECKPOINT 1

Deverá estar implementado:

- processo distribuído;
- socket TCP;
- cliente/servidor;
- serialização;
- framing de mensagens;
- concorrência básica;
- identificação de nós.

Aluno 1 implementa:

network.c

protocol.c

peer.c

Esses três códigos deverão realizar:

- criação do socket;
- bind;
- listen;
- accept;
- connect;
- send;
- recv;
- framing;
- header;
- checksum.

Aluno 2 implementa:

node.c

superpeer.c

Esses dois códigos deverão realizar:

- NodeID;
- configuração;
- identificação do processo;
- registro do nó;
- tabela básica de membros;
- criação do Super Peer.

Itens de verificação:

- conexão TCP funcionando;
- mensagem chega corretamente;
- header é interpretado;
- checksum é validado;
- dois processos podem conversar;
- não pode ocorrer segmentation fault.

7) CHECKPOINT 2

Aluno 1 será responsável por:

- upload
- download
- storage
- chunking
- SHA-256
- LZ4
- parallel transfer

e pipeline:

arquivo

↓

SHA-256

↓

fragmentação

↓

LZ4

↓

checksum

↓

transferência

Aluno 2 é responsável por:

- metadata
- hash table

- ObjectID
- registro dos chunks

Estrutura em C do metadado:

```
typedef struct {
    uint8_t object_id[32];

    char filename[256];

    uint64_t size;

    uint32_t chunk_count;

    uint8_t **chunk_hashes;

    uint32_t version;

    uint32_t owner;
} FileMetadata;
```

Itens de verificação:

./peer upload arquivo.pdf

O sistema deverá produzir:

File: arquivo.pdf

Size: 12458291 bytes

ObjectID:

a7f3...

Chunks: 3

Chunk 0: ...

Chunk 1: ...

Chunk 2: ...

Compression: LZ4

Depois:

./peer download arquivo.pdf

O sistema deverá produzir:

Download completed

SHA-256 verified

8) CHECKPOINT 3

Aluno 1:

membership
heartbeat
suspect
failed
removed

Estados:

ALIVE



SUSPECT



FAILED



REMOVED

Heartbeat: 5 segundos

Timeout: 15 segundos

Aluno 2:

NodeID

successor

predecessor

finger table

lookup

join

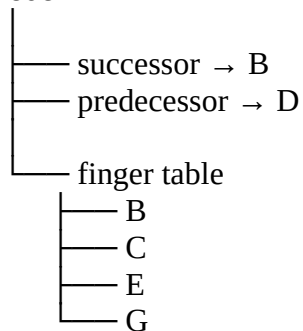
stabilize

notify

fix_fingers

Exemplo:

Node A



Itens de verificação:

Inicialização:

SP1

SP2

SP3

SP4

SP5

Posteriormente:

JOIN SP2

JOIN SP3

JOIN SP4
JOIN SP5

Teste:

lookup <ObjectID>

Saída:

Lookup path:

SP1

↓

SP3

↓

SP5

Owner: SP5

Teste de falha:

kill -9 SP3

Saída:

SP3 SUSPECT

SP3 FAILED

9) **CHECKPOINT 4**

Aluno 1:

ELECTION

OK

COORDINATOR

Fluxo:

Coordinator morreu

↓

timeout

↓

ELECTION

↓

nós de maior prioridade

↓

OK

↓

novo coordenador

↓

COORDINATOR

Aluno 2:

operation log

log index

transaction ID

version

checksum

Exemplo:

LOG #101

PUT fileA

LOG #102
PUT fileB

LOG #103
DELETE fileA

Itens de avaliação:

Início:

SP1

SP2

SP3

SP4

SP4 é coordenador.

Executa: kill -9 SP4

Sistema deve retornar:

detect failure

↓

start election

↓

highest NodeID wins

↓

COORDINATOR

↓

resume operations

Próxima execução:

upload arquivo.pdf

E compara:

metadata SP1

metadata SP2

metadata SP3

Todos precisam estar logicamente consistentes.

10) **CHECKPOINT 5**

Aluno 1: implementa LFU + Transferência.

LFU

cache hit

cache miss

frequency

eviction

Aluno 2: implementa 2PC + IST

PREPARE

↓
YES / NO

↓
COMMIT / ABORT

Funcionamento do IST:

Novo nó: version 1032

Coordenador: 1050

Transferir: 1033 ... 1050 em vez de toda a base.

Isso corresponde diretamente ao mecanismo de Incremental State Transfer descrito no texto.

Itens de avaliação:

Início:

SP1

SP2

SP3

Retorna:

UPLOAD A

UPLOAD B

UPLOAD C

Vamos derrubar: SP2

Enquanto SP2 está fora:

UPLOAD D

UPLOAD E

Reinicia SP2

Após reinício, tem-se:

SP2

↓
JOIN

↓
IST

↓
versions missing

↓
receive logs

↓
READY

Teste de 2PC:

Vamos forçar uma falha durante a fase **PREPARE**.

O sistema deve evitar deixar os participantes em estados inconsistentes.

10) CHECKPOINT 6 – Integração e teste final.

Aluno 1 deve demonstrar:

upload
download
chunks
compression
cache
replica

Aluno 2 deve demonstrar:

Chord
Gossip
Bully
SMR
2PC
IST

Por fim, ambos podem demonstrar juntos:

Integração, Debugging, Documentação e Testes.

Nessa fase eu vou fornecer um conjunto de testes para testar o sistema.

OBSERVAÇÃO IMPORTANTE: SÓ SE AVANÇA PARA O CHECKPOINT SEGUINTE E O ATUAL ESTIVER FUNCIONAL. CASO CONTRÁRIO, NA PRÓXIMA DATA DO CHECKPOINT, O GRUPO DEVERÁ APRESENTAR O CHECKPOINT ANTERIOR.

OBSERVAÇÃO 2: AO FINAL DE CADA CHECKPOINT, O GRUPO DEVERÁ FORNECER UM DOCUMENTO COM A EXPLICAÇÃO DO DESENVOLVIMENTO DAQUELE CHECKPOINT.

Gradação do trabalho:

Checkpoint 1: 1,0 ponto;
Checkpoint 1: 1,0 pontos;
Checkpoint 1: 2,0 pontos;
Checkpoint 1: 2,0 pontos;
Checkpoint 1: 2,0 pontos;
Checkpoint 1: 2,0 pontos;